

COMPETITOR GUIDE · v1.0.0

RANK :: CHALLENGE

React Coding Challenges

10

CHALLENGES

2650

TOTAL POINTS

React

ONLY

A progressive, code-fixing React challenge that takes you from JSX and props all the way to custom hooks and data fetching. Each level unlocks the next. Fix the code, earn the points, climb the developer ranking.

Generated 2026 · RANK::CHALLENGE v1.0

TABLE OF CONTENTS

01	What is Rank Challenge?	3
02	Install Node.js	4
03	Run the Challenge	5
04	The 10 Challenges	6
05	Scoring & Rules	7
06	The Interface & Finishing	8
07	Tips & Quick Reference	9

What is Rank Challenge?

Rank Challenge is a code-fixing competition for **React developers**. Every level hands you a small, broken React component. Your job is to fix it so it behaves correctly — and the platform grades your code automatically.

There are **10 challenges** worth a combined **2,650 points**. They form a single progression: each level you solve unlocks the next, and your total score places you on the developer ranking. The further you get and the fewer hints you use, the higher you rank.

Every challenge is React

The whole track is React — no CSS-only, network, or backend levels. You'll work entirely in `.jsx` files inside an in-browser editor, with a live preview of your component next to it. Topics climb from the fundamentals to real-world hooks:

- **Fundamentals** — JSX syntax, `className`, rendering values, props
- **State & events** — `useState`, controlled inputs, event handlers
- **Rendering patterns** — lists with `key`, conditional rendering
- **Effects & data** — `useEffect`, fetching with `async/await`
- **Advanced** — lifting state up, `useRef`, and building a custom hook



You only need to know React

No security, networking, or server knowledge is required. If you can write a function component and use hooks, you have everything you need.

How grading works

When you press **Submit**, the server analyses your code against a set of checks for that level (for example: "uses `useState`", "reads `e.target.value`", "renders the list with a `key`"). All checks must pass to clear the level and bank the points.

Rank Challenge runs on **Node.js** (which includes `npm`). You need **Node 18 or newer** — the latest **LTS** release is recommended.

Step 1 — Check if you already have it

Open a terminal (Terminal on macOS/Linux, or PowerShell / Command Prompt on Windows) and run:

```
terminal
```

```
node -v
```

```
npm -v
```

If both print a version (and `node -v` is `v18` or higher), you're ready — skip to the next chapter. If you see "command not found" or a version below 18, install or update Node below.

Step 2 — Download & install

RECOMMENDED — OFFICIAL INSTALLER

- Download page (pick **LTS**): <https://nodejs.org/en/download>
- Direct prebuilt installers (Windows `.msi` / macOS `.pkg`): <https://nodejs.org/en/download/prebuilt-installer>

Run the downloaded installer, accept the defaults, then **close and reopen your terminal** and re-run `node -v` to confirm.

ALTERNATIVE — VERSION MANAGER (HANDY FOR MULTIPLE NODE VERSIONS)

- macOS / Linux — **nvm**: <https://github.com/nvm-sh/nvm>
then run `nvm install --lts`
- Windows — **nvm-windows**: <https://github.com/coreybutler/nvm-windows>
then run `nvm install lts`

ALTERNATIVE — PACKAGE MANAGERS

```
terminal
```

```
# macOS (Homebrew)
```

```
brew install node
```

```
# Windows (winget)
```

```
winget install OpenJS.NodeJS.LTS
```

```
# Debian / Ubuntu (via NodeSource — see nodejs.org for the latest)
```

```
sudo apt-get install -y nodejs npm
```



After installing

Always open a **fresh** terminal window so it picks up the new `node` and `npm` commands, then verify with `node -v`.

3

Run the Challenge

With Node installed, you start the platform from the project folder in three commands.

Step 1 — Open the project

```
terminal
```

```
cd path/to/ctf
```

Replace `path/to/ctf` with wherever you unzipped/cloned the project (the folder containing `package.json`).

Step 2 — Install dependencies (first time only)

```
terminal
```

```
npm install
```

This installs both the frontend and backend packages (the project uses npm workspaces). It only needs to be done once, or after the project is updated.

Step 3 — Start it

```
terminal
```

```
npm run dev
```

This launches the frontend (**`http://localhost:5173`**) and the backend (**`http://localhost:3001`**) together. You'll see a line like `Rank Challenge server running on http://localhost:3001`. If port 3001 is busy, the backend automatically moves to the next free port.

Step 4 — Open & log in

Open <http://localhost:5173> in your browser. From the info page click **Continue**, then enter your **6-digit access code**. A successful login starts a **1-hour session**.



Getting your code

Login codes are issued by the 2FA service your organiser runs (on `http://localhost:3000`). Each code is single-use and expires quickly — ask your organiser for a fresh one, then type it on the login screen.



Port already in use?

If something else is holding port 5173, stop it (or close the other dev server) and run `npm run dev` again.

Stopping

Press `ctrl + c` in the terminal running `npm run dev` to stop both servers.

The 10 Challenges

All ten levels are React. Solve them in order — each unlocks the next.

#	TITLE	WHAT YOU'LL FIX	POINTS
01	Broken JSX	<code>className</code> & rendering a variable with <code>{}</code>	100
02	Pass the Props	Reading <code>props</code> in a child component	150
03	Stateful Counter	<code>useState</code> instead of a plain variable	200
04	Controlled Input	<code>e.target.value</code> & <code>preventDefault()</code>	250
05	Render the List	<code>.map()</code> with a <code>key</code> prop	250
06	Fetch on Mount	<code>useEffect</code> + <code>await fetch</code>	300
07	Show and Hide	Conditional rendering with <code>&&</code>	300
08	Lift State Up	Controlled child via <code>props.value</code> / <code>props.onChange</code>	350
09	Focus with a Ref	<code>useRef</code> & <code>ref.current.focus()</code>	350
10	Custom Hook Finale	Extract logic into a <code>use...</code> hook	400



Each level tells you exactly what to do

Every challenge has a description, a list of objectives, and optional hints. Read the objectives — they map directly to the checks the grader runs.

Points

Each level is worth the points shown above, from 100 (Level 01) up to 400 (the finale) — **2,650** in total. Points are banked the moment all of a level's checks pass.

Hints cost points

Every challenge offers a few hints. Revealing a hint subtracts a fixed **hint penalty** from that level's score (it grows with difficulty — roughly 15 points on early levels up to 40 on the finale). Use them only when stuck.

Unlock chain

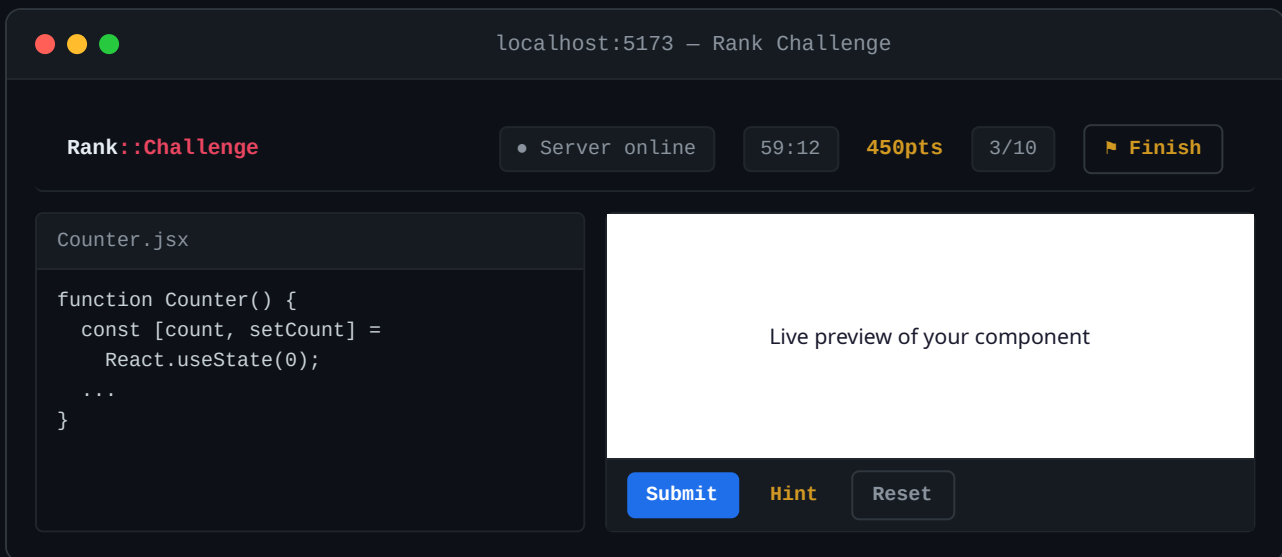
Levels unlock in sequence: clear Level 01 to open Level 02, and so on to the finale. Your progress is saved, so you can close the tab and pick up where you left off (within your session).

Session window

A login is valid for **1 hour**. When the timer in the top bar runs out you're returned to the login screen and need a new code.

Submission limit

Submissions are rate-limited to **20 per minute** to keep things fair — there's no penalty, you just wait a moment if you hit it.



Top bar

- **Server status** — confirms the backend is reachable
- **Session timer** — time left in your 1-hour window
- **Score & progress** — your points and solved/total count
- **Challenge Map** (home icon) — jump back to the level grid
- **Finish** — end your attempt (see below)

Editor & preview

The left panel is the code editor (your `.jsx` file). The right panel shows a **live preview** that re-renders as you type. Use **Submit** to grade, **Hint** to reveal a hint (costs points), and **Reset** to restore the original buggy starter.

Finishing the challenge

When you're done, press **Finish** in the top bar. You'll see a summary of how many levels you solved and your final score, then confirm to end the session. To start a fresh attempt, log in again with a new code.



Your score is your rank

The more levels you clear — and the fewer hints you spend — the higher you place on the developer ranking.

Strategy

- **Read the objectives first.** They mirror exactly what the grader checks.
- **Watch the live preview.** If your component renders correctly there, you're usually close.
- **Save hints for later.** Try to fix yourself first — hints cost points.
- **Submit early, submit often.** Failed checks tell you what's still wrong (within the 20/min limit).
- **Reset if you get lost.** The Reset button brings back the original starter code.

Quick command reference

terminal

```
# check your Node version (need v18+)
node -v

# install dependencies (first run only)
npm install

# start the platform (frontend + backend)
npm run dev

# then open
http://localhost:5173
```

Useful links

- Download Node.js (LTS): <https://nodejs.org/en/download>
- Prebuilt installers: <https://nodejs.org/en/download/prebuilt-installer>
- nvm (macOS/Linux): <https://github.com/nvm-sh/nvm>
- nvm-windows: <https://github.com/coreybutler/nvm-windows>
- React docs: <https://react.dev/learn>